Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom

Disusun oleh:
Mukhammad Raffi Zabra
T1 – 1G
254107020059

POLITEKNIK NEGERI MALANG
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141
TAHUN 2026-2027

C. Langkah Kerja
Langkah 1: Melanjutkan Proyek bank-mini

> ✅ **Checkpoint:** program masih mengompilasi dan berjalan, menampilkan tiga baris - balance: seperti pada akhir Pertemuan 2 (Nadia, Budi, Sari).

Hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P3/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P3 id.ac.polinema.Main
Nadia - balance: 350000.0
Budi - balance: 1000000.0
Sari - balance: 500000.0
```

Gambar 1 hasil dari pertemuan 2

Langkah 2: Menerapkan Encapsulation ke Account
Class account

P3 > id > ac > polinema > Account.java > Account

```java
package id.ac.polinema;

public class Account {

    private String accountNumber;
    private String ownerName;
    private double balance;

    public Account(String accountNumber, String ownerName, double balance){
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = balance;
    }

    public String getAccountNumber(){
        return accountNumber;
    }

    public String getOwnerName(){
        return ownerName;
    }

    public double getBalance(){
        return balance;
    }

    public boolean deposit(double amount){
        if (amount <= 0){
            return false;
        }
        balance += amount;
        return true;
    }

    public boolean withdraw(double amount){
        if (amount <= 0 || amount > balance){
            return false;
        }
        balance -= amount;
        return true;
    }

    public void printInfo(){
        System.out.println(accountNumber + " - " + ownerName + " - Balance: " + balance);
    }
}
```

Gambar 2 Penerapan enkapsulasi pada kode Account.java

Penjelasan : class ini menerapkan prinsip enkapsulasi dengan mengunci atribut accountNumber, ownerName, dan balance menggunakan *access modifier* private. Agar datanya tetap bisa dibaca secara aman, disediakan method *getter* (getAccountNumber, getOwnerName, getBalance). Dan method deposit() dan withdraw() diubah bertipe boolean yang langsung mengecek validasi di awal jika input salah atau saldo kurang, method langsung mengembalikan false tanpa memodifikasi saldo.

Class main

P3 > id > ac > polinema > Main.java > ...

```java
package id.ac.polinema;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Account acc = new Account(accountNumber: "A001", ownerName: "Nadia", balance: 500000);
        acc.withdraw(amount: 150000);
        acc.printInfo();
    }
}
```

Gambar 3 Pengujian objek rekening pada kode Main.java

Penjelasan : class ini sebagai tempat pengujian class Account di sini dibuat objek rekening baru bernama acc dengan identitas "A001", pemilik "Nadia", dan saldo awal 500000 melalui konstruktor. Selanjutnya, dijalankan penarikan saldo sebesar 150000 melalui method withdraw() lalu seluruh informasi akun dicetak memanggil printInfo()

> ✅ **Checkpoint:** program berhasil dikompilasi ulang dan menampilkan `A001 - Nadia - balance: 350000.0`.

> ⚠ **Jika gagal:** apabila muncul galat `constructor Account in class Account cannot be applied to given types`, periksa apakah jumlah dan urutan argumen pada `new Account(...)` sudah sesuai dengan constructor yang tersedia.

> **Catatan.** `accountNumber` sengaja hanya diberi getter, tanpa setter, mengikuti pola atribut read-only dari slide konsep. Nilainya ditetapkan sekali lewat constructor dan tidak pernah berubah lagi.

Hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P3/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P3 id.ac.polinema.Main
A001 - Nadia - Balance: 350000.0
```

Gambar 4 Petunjuk checkpoint pengujian Langkah 2

D. Tugas dan Deliverable
Kumpulkan hal berikut sesuai format yang diminta Dosen:
Screenshot output program setelah Langkah 2.

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P3/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P3 id.ac.polinema.Main
A001 - Nadia - Balance: 350000.0
```

Gambar 5 Output eksekusi Langkah 2 di terminal

Penjelasan :

Tugas mandiri:
Class account

P3 > id > ac > polinema > Account.java > Account > Account(String, String, double, double)

```java
package id.ac.polinema;

public class Account {

    private String accountNumber;
    private String ownerName;
    private double balance;
    private double dailyWithdrawalLimit;

    public Account(String accountNumber, String ownerName, double balance, double dailyWithdrawalLimit){
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = balance;
        this.dailyWithdrawalLimit = dailyWithdrawalLimit;
    }

    public String getAccountNumber(){
        return accountNumber;
    }

    public String getOwnerName(){
        return ownerName;
    }

    public double getBalance(){
        return balance;
    }

    public boolean deposit(double amount){
        if (amount <= 0){
            return false;
        }
        balance += amount;
        return true;
    }

    public boolean withdraw(double amount){
        if (amount <= 0 || amount > balance || amount > dailyWithdrawalLimit){
            return false;
        }
        balance -= amount;
        return true;
    }

    public void printInfo(){
        System.out.println(accountNumber + " - " + ownerName + " - Balance: " + balance);
    }
}
```

Gambar 6 Penambahan atribut dailyWithdrawalLimit pada kode Account.java

Penjelasan : class Account dimodifikasi dengan penambahan satu atribut baru, yaitu private double dailyWithdrawalLimit, yang langsung diinisialisasi melalui parameter konstruktor. Pada method withdraw(), aturan pengecekan diperketat dengan menambahkan kondisi amount > dailyWithdrawalLimit. Jadi penarikan uang sekarang harus memenuhi tiga syarat: jumlahnya lebih dari 0, tidak melebihi saldo, dan tidak boleh melewati batas limit harian

Class main

P3 > id > ac > polinema > Main.java > Main > main(String[])

```java
package id.ac.polinema;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Account limited = new Account(accountNumber: "A005", ownerName: "Fajar", balance: 1000000, dailyWithdr… 200000);
        System.out.println("Withdraw 300000 allowed? "+limited.withdraw(amount: 300000));
        System.out.println("Withdraw 150000 allowed? "+limited.withdraw(amount: 150000));
        limited.printInfo();
    }
}

```

Gambar 7 Pengujian limit penarikan harian pada kode Main.java

Penjelasan : class pengujian ini membuat satu objek rekening baru bernama limited untuk pemilik "Fajar", dengan saldo 1.000.000 dan batas penarikan harian 200.000. Kode kemudian menguji dua kondisi penarikan: percobaan pertama menarik 300.000 yangg melebihi limit saldo harian untuk memastikan sistem mengembalikan nilai false, lalu percobaan kedua menarik 150.000 yaitu dibawah limit harian yang menghasilkan true, sebelum akhirnya memanggil printInfo() untuk melihat sisa saldo akhir.

Hasil

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> javac P3/id/ac/polinema/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO> java -cp P3 id.ac.polinema.Main
Withdraw 300000 allowed? false
Withdraw 150000 allowed? true
A005 - Fajar - Balance: 850000.0
```

Gambar 8 Output pengujian penarikan dengan batasan limit harian di terminal

2. Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan):
(a) mengapa mengembalikan nilai boolean dari deposit()/withdraw() lebih aman dibandingkan tidak memberi tahu pemanggil sama sekali ketika nilainya ditolak?
Jawaban :
Nilai balik boolean memberikan laporan kepastian status transaksi kepada sistem pemanggil apakah operasi benar-benar sukses (true) atau gagal (false). Tanpa nilai balik, pemanggil akan mengasumsikan transaksi selalu berhasil, yang berisiko memicu kesalahan fatal lanjutan seperti tetap mencetak struk penarikan atau melanjutkan transaksi lain padahal saldo fisik sebenarnya tidak berubah

(b) sebutkan satu atribut pada Account yang menurutmu sebaiknya dibuat read-only (getter tanpa setter), dan jelaskan alasannya.
Jawaban :
Atribut accountNumber (nomor rekening) sebaiknya dibuat *read-only* hanya dengan getter tanpa setter. Hal ini dikarenakan nomor rekening berfungsi sebagai identitas unik permanen yang ditetapkan sekali saat pendaftaran akun dan tidak boleh diubah-ubah demi menjaga keutuhan riwayat transaksi dan integritas data perbankan.

HASIL DAN PEMBAHASAN
Praktikum ini berhasil menerapkan prinsip enkapsulasi dengan mengubah atribut pada class Account menjadi private dan menyediakan method *getter* untuk mengakses nilainya secara aman. Method deposit() dan withdraw() kini mengembalikan nilai boolean untuk memvalidasi transaksi di awal sebelum saldo diubah, sehingga penarikan tidak valid langsung ditolak. Pada tugas mandiri, penambahan atribut dailyWithdrawalLimit terbukti sukses menolak penarikan yang melebihi batas limit harian.

KESIMPULAN

1. Enkapsulasi melindungi data rekening dari perubahan liar di luar kelas dengan membatasi akses langsung ke atribut menggunakan hak akses private.
2. Penggunaan nilai kembali boolean memberikan laporan yang jelas apakah transaksi berhasil atau gagal, sehingga saldo rekening tetap aman dan akurat.

Lampiran :
Link repositori github : https://github.com/Rappizr/PPBO/tree/main/P3